



Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

CS7CS4 Machine Learning

Week 4

Shane O'Donnell – 21364002

Dataset 1 ID: # id:2-2-2-1

Dataset 2 ID: # id:2--2-2-1

Introduction

This week's investigation entailed delving into the utilisation of Logistic Regression and k-nearest neighbour models and how these models perform on a given dataset. As well as this, an investigation into how the optimal model specifications can be chosen in terms of regularisation penalty strength and the maximum number of polynomial features to use. These models were assessed using two distinctively different datasets.

Method

(a)

This investigation was split into 2 parts which entailed completing the exercise for two datasets, the datasets can be seen plotted below in figure 1. Dataset 1 does not display a distinct pattern of the behaviour of negative and positive and how they differ, while dataset 2 has a clear boundary on how the behaviour of negative and positive classes differ.

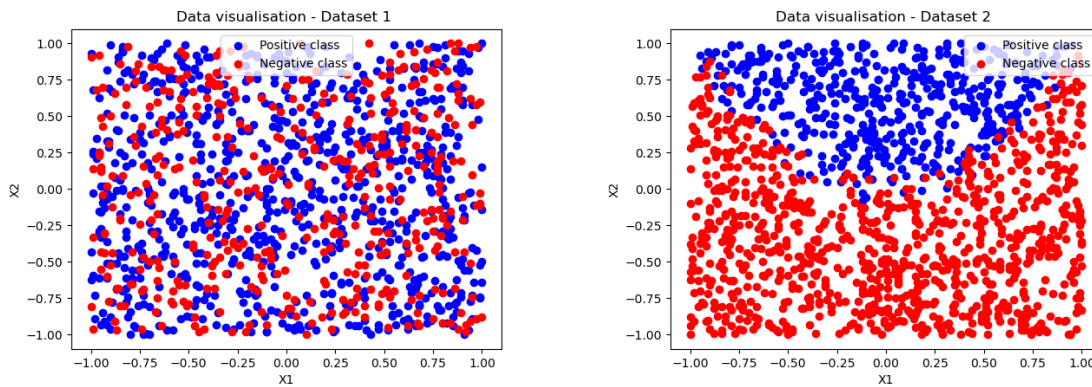


Figure 1: Plot of Datasets provided

The first step of this investigation was to train models using logistic regression with L2 regularisation and a model using k-nearest neighbour. To assess the optimal parameters to use for each model, the parameters were varied for multiple tests and the accuracy was assessed using 5-fold cross validation. With Logistic Regression the probability of a set of features corresponding to a positive value can be calculated with the following formula.

$$p(y = 1|x) = \sigma(x_1w_1 + x_2w_2 + b)$$

$$p(y = 1|x) = \frac{1}{1 + e^{-(x_1w_1 + x_2w_2 + b)}}$$

For the logistic regression model, the regularisation strength penalty [C] was varied as well as the maximum order of polynomial features to utilise. To test every combination between these two terms, a nested for loop was used which iterates through every combination of C and polynomial order. In order to assess the performance of each combination, 5-fold cross validation was used to assess the accuracy of the model using F1 score, this splits the training set into 5 different sections and the model is tested on each of these sections once while being trained on the 4 remaining sections. This makes it

possible to calculate the mean F1 score achieved by each model and the standard deviation of the F1 scores achieved. The formula for F1 score can be seen below. F1 score is the ideal for classification models as it balances the importance of the measuring how often the model is correct for positive classes while also measuring the models ability to find all the positive instances in the model.

$$F1\ Score = \frac{2}{\frac{1}{precision} + \frac{1}{recall}}$$

$$precision = \frac{True\ Positive}{True\ Positive + False\ Positive}$$

$$recall = \frac{True\ Positive}{True\ Positive + False\ Negative}$$

Figure 2 outlines the performance of models with different maximum polynomial feature order and how their accuracy varied when different values of C were applied. From the visual as well as searching the results array, the best performing model has a maximum polynomial order of 1 and a C value of 0.01, this model scored a mean F1 score of 0.79222901 with a standard deviation of +/- 0.001605968. To have a clear view of the top performing polynomial order models, [1,4,11] were plotted alone with the standard deviation displayed as well. It can be seen that many of the models perform equally as well regardless of the polynomial order used but the performance can change drastically depending on the regularisation penalty.

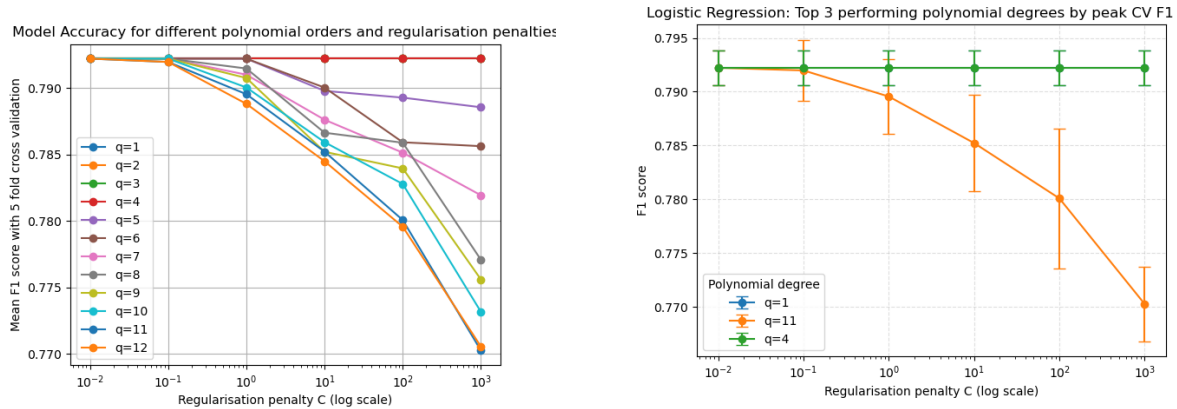


Figure 2: Model F1 scores at different max polynomial orders and regularization penalty

(b)

For the k-nearest neighbour models, the only parameter to vary is the [k] value which assesses how many neighbouring points should be considered to create the class borders. This assessment was conducted in similar fashion by training multiple models with varying k values and assessing the F1 score for each model. The reason the features do not need to be augmented for this model is kNN models can already model nonlinear decision boundaries. By analysing the graph and searching the accuracy results array, the optimal k value was found to be 27 with a mean cross validated F1 score of [0.7827216966] and a standard deviation of +/- [0.004733273992117215].

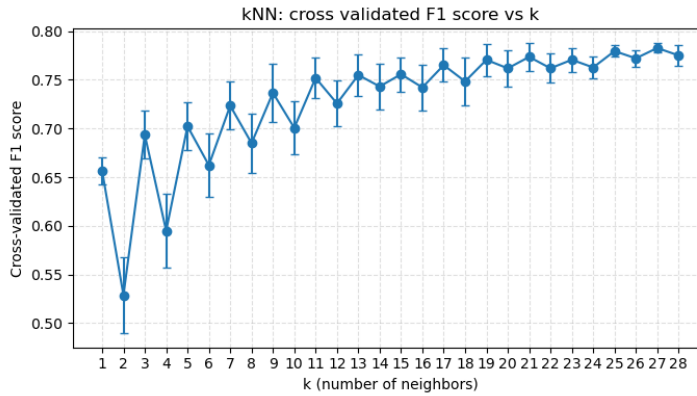


Figure 3: kNN accuracy at different k values - evaluated with 5 fold cross validation

(c)

The next step entailed calculating the confusion matrices for each model using its best performing parameters calculated in the earlier investigation. A confusion matrix summarises the performance of a classification model by comparing the models' predictions with the actual label for each datapoint. predictions are categorised into a table which outlines four types, true positive, true negative, false positive, false negative. Not only does this outline the accuracy of the model but also outlines the nature in which the model mis predict values.

The confusion matrix for the logistic regression model with a regularisation penalty $[C]$ of $[0.01]$ and a maximum polynomial feature order of $[1]$ can be seen outlined below. It can be seen that from the table that the logistic regression model always predicts positive, this is evidence that the logistic regression models with the polynomial orders test can't model the randomness apparent in dataset 1.

Table 1: Confusion Matrix for Logistic Regression model - Dataset 1

	False	True
Negative	95	0
Positive	0	162

Furthermore, the confusion matrix for the k-nearest neighbour model with a k value of $[27]$ can be seen outlined below. Compared to the Logistic Regression model the kNN does not perform much better and shows again that the randomness of the dataset is a struggle to model for these machine learning techniques.

Table 2: Confusion Matrix for kNN model - Dataset 1

	False	True
Negative	94	1
Positive	4	158

As well as this the confusion matrices for the models which choose the most frequent class $[+1]$ and the model which chooses randomly are outlines in table 3 and 4 respectively. It can be seen that the

model which always chooses +1 models the same behaviour as the logistic regression model as well as being very similar to the kNN model.

Table 3: Confusion Matrix for model which chooses most frequent class (+1)

	False	True
Negative	95	0
Positive	0	162

Table 4: Confusion Matrix for model which chooses randomly

	False	True
Negative	52	43
Positive	87	75

In order to better visualise the prediction points, the original data and the predicted points from the test data can be seen plotted for the optimal parameter logistic regression model and the kNN model are displayed below on a scatter plot. By analysing the Logistic Regression models predictions and the full dataset, there is a lack of -1 class predictions due models' inability to model the noisiness of the dataset.

By analysing the predicted points for the kNN model, there is similar behaviour to the Logistic Regression model, although the kNN model possessed the ability to predict some negative class points in areas that are more populated by negative class points, although it is still very minimal and shows that for this type of model most predictions are positive and can not model the noisy dataset well.

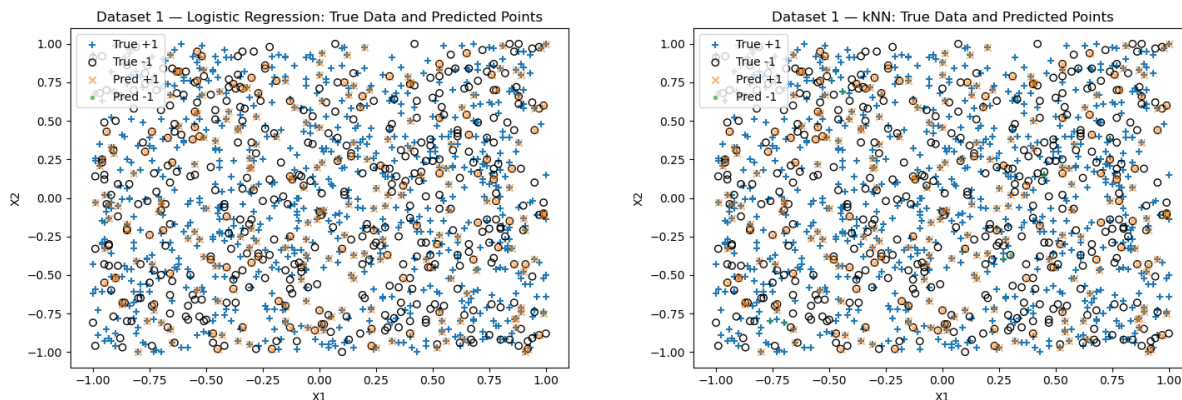


Figure 4: True Data Points plotted with Predicted Points for Logistic Regression and k-nearest neighbour models

(d)

To further the analysis of the performance of each model, a ROC curve was plotted for each model. An ROC curve plots the True Positive Rate and False Positive Rate at varying threshold settings. The threshold setting refers to the level of certainty the model has have to predict for a value to be considered a member of the positive class; by having a lower threshold, this will lead to a greater true

positive rate as well greater false positive rate, while increasing the threshold will lead to a lower false positive rate but also may lead to more true positives being missed. The formulas below outline how the probability of a point being positive is calculated, due to the Logistic regression returning a probability, by lowering the threshold, points can be predicted positive at a lower confidence level.

$$p(y = 1|x) = \sigma(x_1w_1 + x_2w_2 + b)$$

$$p(y = 1|x) = \frac{1}{1 + e^{-(x_1w_1 + x_2w_2 + b)}}$$

The ROC curve for logistic regression and k-nearest neighbour can be seen below; the plot also includes points which correspond to the TPR and FPR achieved for models which always choose the most frequent class, as well as a model which chooses randomly. From the graph, it can be seen that at most threshold values, the models fail to outperform the model which simply chooses the most frequent class or the model which chooses randomly. All four approaches have very similar AUC values, the AUC value refers to the area under the ROC curve of each model. This analysis shows that for a dataset with such ambiguous boundaries which can be seen in figure 1, the models chosen fail to model the intricacies and “randomness” of the plots to the point in which even a trained model doesn’t outperform picking randomly.

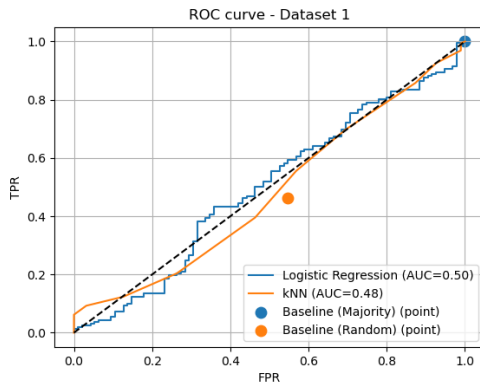


Figure 5: ROC curve for Logistic Regression and kNN and points outlining 2 baseline models (pick randomly and choose most frequent class)

Dataset 2

(a)

In order to further the investigation, the completed for the first dataset were repeated for the second dataset, visually from figure 1 it can be seen that the border between classes is more distinct for the second dataset. Keeping this in mind, the investigation into the optimal parameters for logistic regression when picking the maximum polynomial feature order and the regularisation strength penalty was repeated for this dataset and scoring the models on their F1 score achieved, the findings are outlined in figure 6.

It can be seen from the visualisations that the models perform significantly better on this dataset due to its clear boundary between classes, making it easier to model a distinct boundary line using logistic regression. From the plot as well as searching the saved F1 scores it was found that the best model

parameters was the model with a maximum polynomial order of 3 and a regularisation penalty of 10 achieving a mean 5 fold cross validation F1 score of 0.9457430964645172 with a standard deviation of ± 0.00969695002395887 . It can be also seen that any model which possessed a polynomial feature (ie. Any model except for $q=1$) was able to achieve a respectable F1 score, most notably at higher regularisation penalty strengths

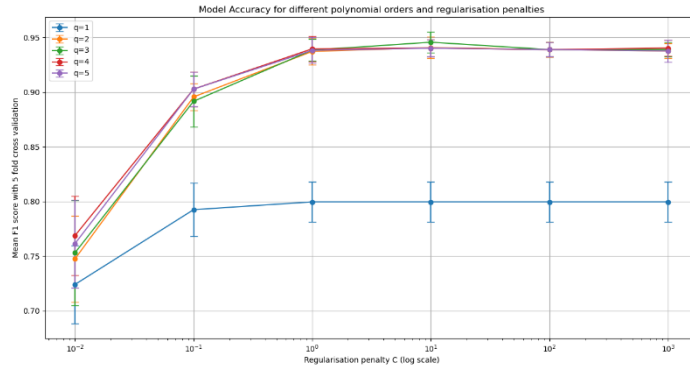


Figure 6: Model F1 scores at different max polynomial orders and regularization penalty

(b)

To further this investigation the kNN model parameters were also evaluated to find the optimal number of neighbouring points to consider. kNN models create boundaries between classes by assessing each point in the training dataset and checking the classes of “k” neighbours closest to them. From this they assign that area to whatever the majority class is.

The results for this investigation using the second dataset can be seen in figure 6 below. As seen from the mean F1 scores calculated with 5 fold cross validation, the model achieves a score above 0.90 for all number of neighbours but performs most notably well for when neighbours was set 6 or 26. By searching the F1 scores stored, the best mean score was associated with a kNN model with 26 neighbours which achieved a mean F1 score of 0.9446725079339469 with a standard deviation of ± 0.011123369527209326 .

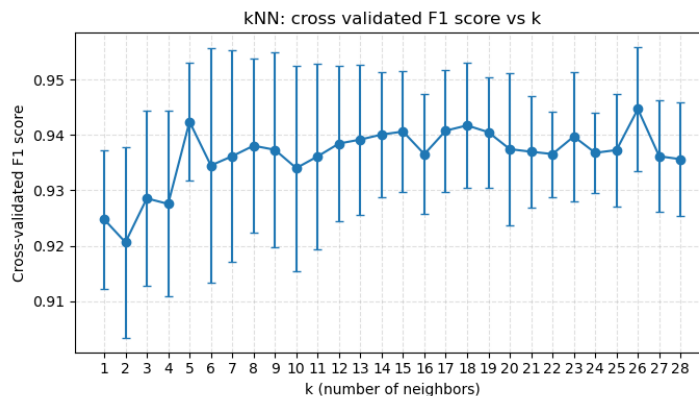


Figure 7: k-nearest neighbour mean F1 score assessed with cross validation

(c)

The confusion matrices were then calculated for each type of model using their best performing parameters. Compared to the confusion matrices from the models trained and tested on dataset 1, the dataset 2 models perform significantly better. The logistic regression model correctly predicts the correct class except for the presence of 8 False negatives and 9 False positives.

Table 5: Confusion Matrix for Logistic Regression model - Dataset 2

	False	True
Negative	8	184
Positive	9	88

The confusion matrix for the 26 nearest neighbours model achieved similar results to the logistic regression model but with less False Negative and False Positives.

Table 6: Confusion Matrix for kNN model - Dataset 2

	False	True
Negative	5	187
Positive	6	91

Table 7: Confusion Matrix for model which chooses most frequent class (-1)

	False	True
Negative	0	192
Positive	97	0

Table 8: Confusion Matrix for model which chooses randomly

	False	True
Negative	89	103
Positive	46	51

Similar to the first dataset, the predicted points were plotted with the original dataset to further understand the model's behaviour. From the logistic regression model scatter plot, it can be seen that the

Logistic Regression model with the calculated optimal parameter is able to model the behaviour of the dataset due to the clear boundary apparent in the dataset.

This can also be seen in the kNN models scatter plot, the predictions model the same behaviour as the training data.

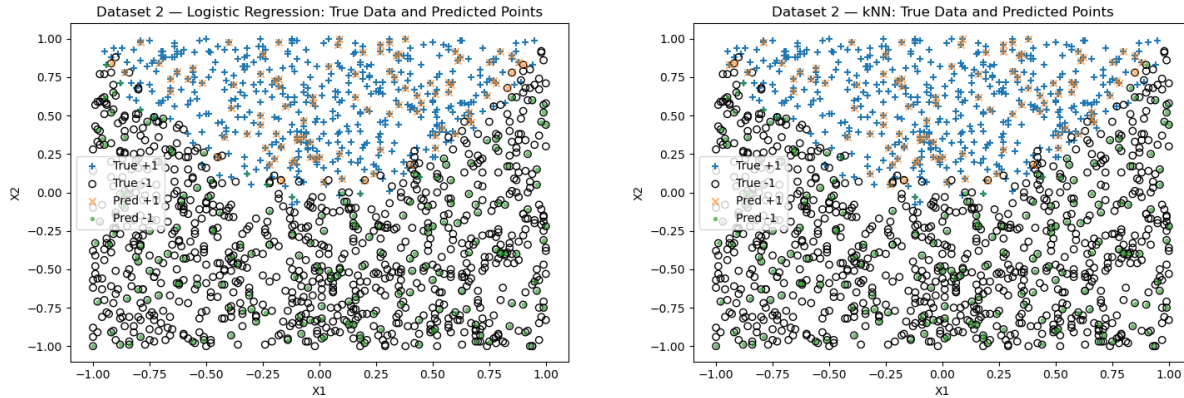


Figure 8: True Data Points plotted with Predicted Points for Logistic Regression and k-nearest neighbour models

(d)

To assess the performance of the models further, the ROC curve for these models as well as the baseline models which guess randomly and the other always chooses the most frequent class. From the plot it can be inferred that the models trained and tested on dataset 2 perform extremely well with AUC values of 0.99 and a sharp increase on the y axis (TPR) compared to the ROC curve for the models trained and tested on the first dataset. The point describing the model which always picks the most frequent class is apparent at the [0,0] point, this is due to the most frequent class being negative for this dataset so [-1] is always predicted. Therefore, the True Positive and False Positive rate will be 0 as positive is never predicted. The model which chooses randomly performs comparably on both datasets.

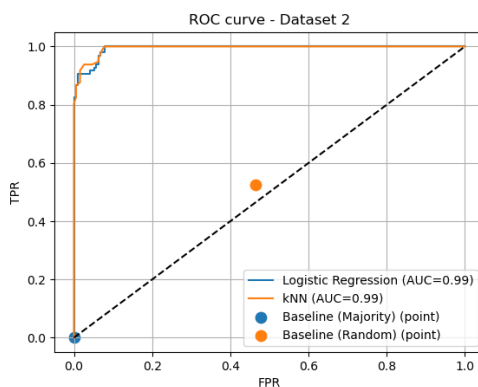


Figure 9: ROC curve for Logistic Regression, kNN, most frequent class model, and random choice model

From the investigation conducted on dataset 2 it can be seen that the models perform significantly better on this dataset compared to the first due to the behaviour of each classes and the direct correlation

on how the value of each feature can influence what class the feature couple belongs to. This statement is backed up by the performance of the models outlined by their F1 score, the confusion matrices displaying less false predictions and the ROC curves being closer to the ideal, the ROC curves for dataset 2 can be seen to have AUC values of 0.99 for both the Logistic Regression model and the kNN model.

Conclusion

To conclude, this investigation offered great insight into how the ideal parameters can be chosen for a model in the training process as well offering experience in training models for two distinctly different datasets, one dataset being incredibly noisy and without a clear pattern and one offering a distinct boundary between both classes.

Code

```
import matplotlib.pyplot as plt

from mpl_toolkits.mplot3d import Axes3D

import numpy as np

import pandas as pd

from sklearn.linear_model import LogisticRegression

from sklearn.preprocessing import PolynomialFeatures

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score

from sklearn.model_selection import cross_val_score

from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import confusion_matrix

from sklearn.metrics import roc_curve, auc

from sklearn.metrics import f1_score

from sklearn.model_selection import StratifiedKFold

from sklearn.metrics import accuracy_score

import matplotlib.pyplot as plt

from matplotlib.lines import Line2D

## data id
```

```

#Dataset 1:
## # id:2-2-2-1 ##

#Dataset 2:
## ## id:2--2-2-1

#function to read in data and make initial plot- return train test splits
def read_in_data(dataset,dataset_num):

    #use week4.csv for the first dataset

    #load in data
    df = pd.read_csv(f'{dataset}.csv', header=None)
    print(df.head())
    X1 = df.iloc[:,0].values
    X2 = df.iloc[:,1].values
    X = np.column_stack([X1, X2])
    y = df.iloc[:,2].values
    print(X.shape, y.shape)

    #split data according to y class label +1 and -1
    X1_pos = X1[y==1]
    X2_pos = X2[y==1]
    X1_neg = X1[y==-1]
    X2_neg = X2[y==-1]

    #visualised data as scatter plot
    plt.figure()
    plt.scatter(X1_pos,X2_pos,c='blue',label='Positive class')
    plt.scatter(X1_neg,X2_neg,c='red',label='Negative class')
    plt.xlabel('X1')
    plt.ylabel('X2')
    plt.title(f'Data visualisation - Dataset {dataset_num} ')
    plt.legend()

```

```

plt.show()

X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2, random_state=42)

#split data into train test sets 0.8:0.2
#X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

return X, y, X_train, X_test, y_test, y_train

def log_reg_model_train_eval(X_train, y_train):
    q_val = [1,2,3,4,5]
    c_val = [0.01,0.1,1,10,100,1000]
    #store f1 scores and std_dev
    acc_results_f1 = np.zeros((len(q_val), len(c_val)))
    std_dev_results_f1 = np.zeros((len(q_val), len(c_val)))

    #5 fold data
    #cv = StratifiedKfold(n_splits=5, shuffle=True, random_state=42)

    for i, q in enumerate(q_val):
        for j, c in enumerate(c_val):
            #fold_scores_f1 = []

            ##manual cross val
            #for train_index, valid_index in cv.split(X_train, y_train):
            #    x_trn, x_val = X_train[train_index], X_train[valid_index]
            #    y_trn, y_val = y_train[train_index], y_train[valid_index]
#
            poly = PolynomialFeatures(degree=q)
            x_trn_poly = poly.fit_transform(X_train) # fit on fold-train
            #x_val_poly = poly.transform(x_val)

```

```

#f1_scores = cross_val_score()

#

logRegModel = LogisticRegression(penalty="l2", C=c, solver='liblinear', max_iter=1000)

# logRegModel.fit(x_trn_poly, y_trn)

# y_val_pred = logRegModel.predict(x_val_poly)

fold_scores_f1 = cross_val_score(logRegModel,x_trn_poly,y_train,cv=5,scoring='f1')

# #calculate f1 score - more appropriate for classification

# fold_scores_f1.append(f1_score(y_val, y_val_pred, pos_label=1))

# #print(f1_score)


acc_results_f1[i, j] = np.mean(fold_scores_f1)

std_dev_results_f1[i, j] = np.std(fold_scores_f1)

#print(acc_results_f1)

#print(std_dev_results_f1)


#find the best f1 score achieved and the q and c value associated

max_idx = np.unravel_index(np.argmax(acc_results_f1, axis=None), acc_results_f1.shape)

best_q = q_val[max_idx[0]]

best_c = c_val[max_idx[1]]

best_accuracy = acc_results_f1[max_idx]

print(f"Best polynomial order q: {best_q}, Best regularisation penalty c: {best_c}, with F1 score:
{best_accuracy}, with a standard deviation of {std_dev_results_f1[max_idx]}")


##plot f1 score with for each q/c partnership

for i, q in enumerate(q_val):

    #plt.plot(c_val, acc_results_f1[i, :], marker='o', label=f'q={q}')

    plt.errorbar(c_val, acc_results_f1[i, :], yerr=std_dev_results_f1[i,:], fmt="-o", capsize=5, label=f'q={q}')

plt.xscale('log')

plt.xlabel('Regularisation penalty C (log scale)')

plt.ylabel('Mean F1 score with 5 fold cross validation')

```

```

plt.title('Model Accuracy for different polynomial orders and regularisation penalties')
plt.legend()
plt.grid()
plt.show()

return best_q, best_c, acc_results_f1, std_dev_results_f1, q_val, c_val

#TODO - make another plot with best performers and include errors bars because it'll be less noisy now
def plot_best_log_regs(acc_results, std_dev_results, q_vals, c_vals, best_num=3):
    acc_per_q=acc_results.max(axis=1)
    top_acc_q = np.argsort(acc_per_q)[-best_num:][::-1]

    plt.figure()
    #for i in top_acc_q:
    q = q_vals[0]
    mean_acc_vals = acc_results[0,:]
    std = std_dev_results[0,:]
    plt.errorbar(c_vals,mean_acc_vals,yerr=std,fmt='-o',capsize=5,label=f'q={q}')

    #q = q_vals[10]
    #mean_acc_vals = acc_results[10,:]
    #std = std_dev_results[10,:]
    #plt.errorbar(c_vals,mean_acc_vals,yerr=std,fmt='-o',capsize=5,label=f'q={q}')

    q = q_vals[1]
    mean_acc_vals = acc_results[3,:]
    std = std_dev_results[3,:]
    plt.errorbar(c_vals,mean_acc_vals,yerr=std,fmt='-o',capsize=5,label=f'q={q}')

    q = q_vals[2]
    mean_acc_vals = acc_results[3,:]
    std = std_dev_results[3,:]

```

```
plt.errorbar(c_vals,mean_acc_vals,yerr=std,fmt='-o',capsize=5,label=f'q={q}')
```

```
    plt.plot(c_vals,mean_acc_vals,marker='o,label')
plt.xscale('log')
plt.xlabel('Regularisation penalty C (log scale)')
plt.ylabel('F1 score')
plt.title(f'Logistic Regression: Top {best_num} performing polynomial degrees by peak CV F1 score')
plt.grid(True, linestyle='--', alpha=0.4)
plt.legend(title='Polynomial degree')
plt.tight_layout()
plt.show()
```

#(b) - train a kNN classifier and use cross val to find best k val

```
def knn_model_train_eval(X_train, X_test, y_train, y_test,X,y):
    k_val = [1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21,22,23,24,25,26,27,28]
    #create array to store accuracy results
    knn_acc_results = np.zeros(len(k_val))
    knn_std_dev_results = np.zeros(len(k_val))
    for k in k_val:
        #iterate through the k values chosen to test
        knnModel = KNeighborsClassifier(n_neighbors=k)
        #fit model to train data
        #knnModel.fit(X_train, y_train)
        #use cross val to get accuracy - could change this metric in the future
        scores = cross_val_score(knnModel, X_train, y_train, cv=5, scoring='f1')
        #get std dev of accuracy scores
        std_dev = np.std(scores)
        knn_std_dev_results[k_val.index(k)] = std_dev
        knn_acc_results[k_val.index(k)] = np.mean(scores)

plt.figure(figsize=(7, 4))
```

```

plt.errorbar(k_val, knn_acc_results, yerr=knn_std_dev_results, fmt='-o', capsize=3)
plt.xlabel('k (number of neighbors)')
plt.ylabel('Cross-validated F1 score')
plt.title('kNN: cross validated F1 score vs k')
plt.xticks(k_val)
plt.grid(True,linestyle='--', alpha=0.4)
plt.tight_layout()
plt.show()

```

```

#find the best accuracy score in the 2d array and print the k val abd the accuracy

```

```

best_k_index = np.argmax(knn_acc_results)
best_k = k_val[best_k_index]
best_k_accuracy = knn_acc_results[best_k_index]
best_k_std_dev = knn_std_dev_results[best_k_index]
print(f"Best k value for kNN: {best_k}, with accuracy: {best_k_accuracy} +/- {best_k_std_dev}")

```

```

return best_k

```

```

def train_best_params_log_reg(X_train,X_test,y_train,y_test,best_q,best_c):

```

```

    poly = PolynomialFeatures(degree=best_q)
    x_train_poly = poly.fit_transform(X_train)
    x_test_poly = poly.transform(X_test)
    model = LogisticRegression(penalty = "l2", C=best_c, solver='liblinear', max_iter=1000)
    model.fit(x_train_poly,y_train)
    y_pred = model.predict(x_test_poly)
    return y_pred, model, x_test_poly

```

```

def train_best_params_knn(X_train,X_test,y_train,y_test,best_k):

```

```

    model = KNeighborsClassifier(n_neighbors=best_k)
    model.fit(X_train,y_train)
    y_pred = model.predict(X_test)
    return y_pred , model

```



```

#calculate confusion matrices for log and knn models with best parameters
def confusion_matrices(y_test, y_pred_log, y_pred_knn):
    cm_log = confusion_matrix(y_test, y_pred_log)
    cm_knn = confusion_matrix(y_test, y_pred_knn)
    return cm_log, cm_knn

#create confusion matrices for baseline classifiers (one that always predicts the most popular class in the dataset)
#(one that chooses randomly (1 or -1))
def baseline_model_conf_mat(y_train,y_test):
    #find most popular class
    if(np.sum(y_train == 1) >= np.sum(y_train == -1)):
        most_freq_class = 1
    else:
        most_freq_class = -1
    baseline_pred = np.full(y_test.shape, most_freq_class)

    #random number gen
    rng = np.random.default_rng(seed=42) # for reproducibility
    baseline_pred_random = rng.choice([-1, 1], size=y_test.shape)

    #compute confusion matrix
    cm_majority = confusion_matrix(y_test, baseline_pred)
    cm_random = confusion_matrix(y_test, baseline_pred_random)

    return cm_majority, cm_random

def plot_roc(y_test, y_score=None, conf_matrix=None, label='Model'):
    #this will plot the log reg and knn
    if y_score is not None:
        y_true = np.where(y_test == -1, 0, y_test)
        fpr, tpr, _ = roc_curve(y_true, y_score)
        plt.plot(fpr, tpr, label=f'{label} (AUC={auc(fpr,tpr):.2f})')

```

```

#this will plot single point for the 2 baseline models

elif conf_matrix is not None:

    TN, FP, FN, TP = conf_matrix.ravel()

    FPR = FP / (FP + TN) if (FP + TN) else 0.0

    TPR = TP / (TP + FN) if (TP + FN) else 0.0

    plt.scatter(FPR, TPR, s=80, label=f'{label} (point)')


#plot og dataset with the y_pred values with it according to report brief
#logic taken from the wk2 assignemtn

def plot_true_vs_pred(X, y, X_plot, y_pred, title="Predictions over True Data"):

    X1, X2 = X[:, 0], X[:, 1]

    X1_pos, X2_pos = X1[y == 1], X2[y == 1]

    X1_neg, X2_neg = X1[y == -1], X2[y == -1]


    plt.figure(figsize=(7, 5))

    plt.scatter(X1_pos, X2_pos, marker='+', label='True +1')

    plt.scatter(X1_neg, X2_neg, facecolors='none', edgecolors='k', label='True -1')

    #prediction data

    plt.scatter(X_plot[y_pred == 1, 0], X_plot[y_pred == 1, 1], marker='x', label='Pred +1', alpha=0.6)

    plt.scatter(X_plot[y_pred == -1, 0], X_plot[y_pred == -1, 1], marker='.', label='Pred -1', alpha=0.6)

    plt.xlabel('X1')

    plt.ylabel('X2')

    plt.title(title)

    plt.legend()

    plt.tight_layout()

    plt.show()


def main():

```

```

print("=====dataset 1=====")

#read in data and get train test splits
X, y, X_train, X_test, y_test, y_train = read_in_data('week4',1)


#train and evaluate logistic regression model
best_q, best_c, acc_results , std_dev_results, q_vals, c_vals = log_reg_model_train_eval(X_train, y_train)


#plot best performers
plot_best_log_regs(acc_results, std_dev_results, q_vals, c_vals, best_num=3)


#train and evaluate kNN model
best_k = knn_model_train_eval(X_train, X_test, y_train, y_test,X,y)


#call functions to train best models for both log and knn
y_pred_log, logModel, x_test_poly = train_best_params_log_reg(X_train,X_test,y_train,y_test,best_q,best_c)
y_pred_knn, knnModel = train_best_params_knn(X_train,X_test,y_train,y_test,best_k)


#plot y_pred with original data
#overlay predictions on original data
plot_true_vs_pred(X, y, X_test, y_pred_log, title="Dataset 1 — Logistic Regression: True Data and Predicted Points")

plot_true_vs_pred(X, y, X_test, y_pred_knn, title="Dataset 1 — kNN: True Data and Predicted Points")


#get confusion matrices for both
cm_log, cm_knn = confusion_matrices(y_test, y_pred_log, y_pred_knn)
print(f"Confusion matrix for Logistic Regression model:\n{cm_log}")
print(f"Confusion matrix for kNN model:\n{cm_knn}")


#get confusion matrices for baseline tests

```

```

cm_most_common, cm_random = baseline_model_conf_mat(y_train,y_test)

print(f"confusion matrix for most frequenct class: \n{cm_most_common}")
print(f"confusion matrix for random choice: \n{cm_random}")

#get class probabilty
y_score_log = logModel.predict_proba(x_test_poly)[: , 1]
y_score_knn = knnModel.predict_proba(X_test)[: , 1]

# === plot ===
plt.figure()
plot_roc(y_test, y_score=y_score_log, label='Logistic Regression')
plot_roc(y_test, y_score=y_score_knn, label='kNN')
plot_roc(y_test, conf_matrix=cm_most_common, label='Baseline (Majority)')
plot_roc(y_test, conf_matrix=cm_random, label='Baseline (Random)')
plt.plot([0,1],[0,1], 'k--')
plt.xlabel('FPR')
plt.ylabel('TPR')
plt.title('ROC curve - Dataset 1')
plt.legend(loc='lower right')
plt.grid()
plt.show()

#=====dataset 2=====
print("=====dataset 2=====")
X, y, X_train, X_test, y_test, y_train = read_in_data('week4_dataset2',2)
best_q, best_c, acc_results , std_dev_results, q_vals, c_vals = log_reg_model_train_eval(X_train, y_train)

plot_best_log_regs(acc_results, std_dev_results, q_vals, c_vals, best_num=3)

best_k=knn_model_train_eval(X_train, X_test, y_train, y_test,X,y)

```

```

#call functions to train best models for both log and knn
y_pred_log, logModel, x_test_poly = train_best_params_log_reg(X_train,X_test,y_train,y_test,best_q,best_c)
y_pred_knn, knnModel = train_best_params_knn(X_train,X_test,y_train,y_test,best_k)

#plot y_pred with original data
plot_true_vs_pred(X, y, X_test, y_pred_log, title="Dataset 2 — Logistic Regression: True Data and Predicted Points")

plot_true_vs_pred(X, y, X_test, y_pred_knn, title="Dataset 2 — kNN: True Data and Predicted Points")

#get confusion matrices for both
cm_log, cm_knn = confusion_matrices(y_test, y_pred_log, y_pred_knn)
print(f"Confusion matrix for Logistic Regression model:\n{cm_log}")
print(f"Confusion matrix for kNN model:\n{cm_knn}")

cm_most_common, cm_random = baseline_model_conf_mat(y_train,y_test)

print(f"confusion matrix for most frequenct class: \n{cm_most_common}")
print(f"confusion matrix for random choice: \n{cm_random}")

#get class probabiltiy
y_score_log = logModel.predict_proba(x_test_poly)[:, 1]
y_score_knn = knnModel.predict_proba(X_test)[:, 1]

# === plot ===
plt.figure()
plot_roc(y_test, y_score=y_score_log, label='Logistic Regression')
plot_roc(y_test, y_score=y_score_knn, label='kNN')
plot_roc(y_test, conf_matrix=cm_most_common, label='Baseline (Majority)')
plot_roc(y_test, conf_matrix=cm_random, label='Baseline (Random)')
plt.plot([0,1],[0,1],'k--')
plt.xlabel('FPR')
plt.ylabel('TPR')

```

```
plt.title('ROC curve - Dataset 2')  
plt.legend(loc='lower right')  
plt.grid()  
plt.show()  
  
if __name__ == "__main__":  
    main()
```

